Convenient Algebraic Programming with Coercive Subtyping

Henry Blanchette[1*] and Aaron Stump[2*]

[1]University of Maryland.
[2]Boston College.

*Corresponding author(s). E-mail(s): blancheh@umd.edu;
stumpaa@bc.edu;

Abstract

This paper presents a system for more convenient algebraic programming, where datatypes are defined as least fixed points of covariant signature functors, and recursive functions are defined as algebras. This approach requires rolling and unrolling fixed points, and conversion of algebras to functions that recurse over data. To alleviate the burden of these extra operations, in this paper we propose to insert them automatically, using coercive subtyping. The type checker generates subtyping constraints, which are then solved with the help of a custom logic programming engined called Chronolog. The subtyping axioms are given to Chronolog as rules, which uses them to solve the subtyping constraints. To avoid infinite applications of rules for unrolling signature functors, Chronolog provides a mechanism for suspending and resuming certain forms of constraints. The subtyping axioms presented to Chronolog are an equivalent algorithmic version of the usual declarative rules for algebraic programming.

Keywords: strong functional programming, type-based termination

1 Introduction

Strong functional programming is a form of pure functional programming where all functions are statically required to terminate on all inputs. One way to achieve this is through an algebraic approach: datatypes are least fixed points μF of signature functors F, and recursive functions are least fixed points $(\!|a|\!)$, called catamorphisms, of algebras a. The functor F describes one layer of the data, and μF then describes

data consisting of any finite number of layers. We may unroll μF to $F\ \mu F$, and roll it back again. Algebras interpret the operations declared for one layer of the data, and catamorphisms apply algebras across all the layers. There is no other mechanism for recursion or looping in the language, leading to a strong guarantee: all functions written in this style terminate.

One burden is the need to include calls to roll and unroll functors, and to $(\!|-|\!)$, in code. This adds clutter, compared to regular functional programming. In this paper, we address this problem by automatically inserting those coercions wherever they are needed in code. The resulting code looks indistinguishable from regular pure-functional code, and yet falls within the algebraic style, thus assuring termination. To insert these calls automatically, we turn to *coercive subtyping*. The type checker generates subtyping constraints that need to hold, in order for the term to be well-typed. The constraints are processed by a constraint solver, generating coercions that are then inserted at the places where the subtyping constraints were generated.

To implement this, we make use of a custom logic-programming engine called Chronolog. The subtyping axioms are given to Chronolog as rules. Chronolog then uses standard SLD resolution to search for proofs of the constraints, instantiating type variables during proof search via unification. The rules for rolling and unrolling signatures would, if handled naively, lead to diverging proof search. Instead, Chronolog provides a mechanism to indicate that certain problematic constraints should be suspended. These suspended constraints may be resumed if they are instantiated during solving of other constraints. If not, an outer loop of our algorithm handles them by simply equating the lower and upper bound of the subtyping constraints. We prove that solving in this manner terminates.

Standard subtyping rules generally are not suitable for logic programming, due to the presence of a transitivity rule, which nondeterministically guesses B with $A <: B$ and $B <: C$, to solve $A <: C$. We identify (Section 6) an algorithmic set of subtyping rules, without transitivity, and prove it equivalent in Agda to the declarative rules. This both ensures we can use logic programming (with our algorithmic rules) to solve these subtyping constraints, and also assures us that the algorithmic rules are indeed correct. To summarize the contributions:

- We propose a new approach to total algebraic programming, where calls for rolling and unrolling signature functors and turning algebras into catamorphisms are inferred by coercive subtyping.
- We realize this approach with constraint-generating typing, and a constraint solver based on logic programming, modified to avoid infinite unrolling of functors. We prove termination of this algorithm.
- We propose an algorithmic subtyping system, and prove it equivalent to the declarative one in Agda.

2 Syntax

TODO Henry: describe R_f notation

Figure 1 presents the syntax for the language $\mathcal{L}_{cata}$, through which we study coercion inference for algebraic programming. We use notation $\overline{e\ \oplus}$ where $\oplus$ is a syntactic

operator, to indicate a $\oplus$-separated list of expressions e. Datatypes D are described as least fixed points of signature functors F, which we take to be polynomial functors for simplicity. Types feature least fixed points μD of signature functors (referenced by datatype name D), as well as usual function types $A \to B$. Types $D \Rightarrow B$ are the types for algebras over μD computing a value of type B, the carrier of the algebra. Type variables R are introduced in the bodies of algebras.

Terms include the usual constructs of simply typed λ-calculus. There are also constructor applications $k(\overline{a,})$, and simple pattern-matching with notation $\gamma a.\{\overline{k(\overline{x,}) \mapsto b\ |}\}$, where the value of a is matched against constructor patterns $k(\overline{x,})$. For totality, all constructors for a's datatype must be covered. We distinguish applications $c(a)$ of coercions c, which include rolling, unrolling, and catamorphizing. Coercions will be explained with their typing rules in Section 5.

Algebras are denoted $\omega f(x).b$. We here use Mendler-style algebras [1]. With this style, an algebra is presented with a function f from R to the carrier B of the algebra. R is an abstract type introduced in the algebra. The input x has type $D\ R$, where D is the signature functor of the algebra. The body $b : B$ may pattern-match on x and apply f to subdata of type R that are revealed. One benefit over conventional F-algebras is that calling f to recurse makes algebras appear very similar to conventional recursive functional programs, albeit with more abstract typing.

(Datatype names)	D	$\in \{Nat, NatList, \dots\}$
(Constructor names)	k	$\in \{Zero, Suc, Nil, Cons, \dots\}$
(Term variables)	x	$\in$ **TermVars**
(Recursion type variables)	R	$::= R_f$
(Type variables)	X, Y	$\in$ **TypeVars**
(Contexts)	Γ	$::= \cdot \mid \Gamma, (x : A) \mid \Gamma, (R_f : *)$
(Datatype signature functor)	F	$::= \mu X.\overline{k(\overline{P(X)\ \times})\ +}$
(Constructor parameter types)	$P(X)$	$::= X \mid D\ P(X) \mid \mu D$
(Types)	A, B	$::= X \mid R \mid \mu D \mid D\ A \mid A \to B \mid D \Rightarrow B$
(Terms)	a, b, f	$::= x \mid k(\overline{a,}) \mid \lambda(x : A).b \mid (f\ a) \mid \omega f(x).b \mid$ $\gamma a.\{\overline{k(\overline{x,}) \mapsto b\ |}\} \mid let\ x : A = a\ in\ b \mid c - a$
(Coercions)	c, c_1, c_2	$::= cata\ c_1\ c_2 \mid roll\ c \mid unroll\ c \mid cov[D]\ c \mid$ $id[\mu D] \mid arr\ c_1\ c_2 \mid alg[D]\ c \mid id[R]$

Fig. 1: Syntax of $\mathcal{L}_{cata}$

A datatype is defined by a tuple $\langle D, F\rangle$ where D is its name and F is its signature functor. This approach was proposed and developed in early works like [2, 3]. For

examples presented in this paper, we define the following set of datatypes.

$$\mathscr{D} = \{\langle Nat, \mu X.Zero + Suc(X)\rangle, \\ \langle NatList, \mu X.Nil + Cons(\mu Nat \times X)\rangle\}$$

TODO Henry: note that throughout the rest of the document, we implicitly restrict variables to range over their appropriate syntactic kinds as defined in the grammar above, for example R only ranges over R_f. **TODO** Henry: note that type variables only appear in constraints, but all types inferred and that you can write in the langauge should be concrete (variable-free) **TODO** Henry: make sure that all overlines use proper separator

3 Motivating examples

```
sub : μ Nat → Nat ⇒ μ Nat =
  λ m . ω sub'(n) . γ n .
    { Zero → m
    | Suc(n') → γ (unroll id) - (sub' n') .
        { Zero → (roll id) - (Zero)
        | Suc(m') → m' } }

subs : NatList ⇒ μ NatList → μ NatList =
  ω subs'(l1) . λ l2 . γ l1 .
    { Nil → Nil
    | Cons(h1, t1) → γ (unroll id) - (l2) .
        { Nil → (roll id) - Nil
        | Cons(h2, t2) →
          (roll id) -
            (Cons((cata id id) - sub h1 h2, subs' t1 t2)) } }
```

Fig. 2: Algebraic implementation of `sub` and `subs`, with explicit coercions.

Figure 2 shows code with explicit coercions, for two functions. The `sub` function subtracts two natural numbers, and `subs` proceeds through a two lists of numbers, subtracting corresponding elements. So subtracting the list (using Haskell syntax) $[1, 2, 3]$ from $[8, 9, 10]$ produces $[7, 7, 7]$, for example. Let us see how these use the different kinds of coercions. The `sub` function takes in the minuend `m` and returns an algebra, which can be used to recurse through the subtrahend `n`. The code matches (with γ) on `n`, returning `m` in the `Zero` case. In the `Suc(n')` case, we recursively subtract `n'` (from `m`). Since our algebras are Mendler-style, the type of `sub'` is `R → μ Nat`, where `R` is the abstract type introduced by the algebra. The input n has type `Nat R`, and so its subdatum `n'` has type `R`. So the call `sub' n'` is well-typed. To pattern-match on the output of this call requires unrolling `μ Nat` with `unroll id`. The `id` is the identity

coercion, indicating that the subtyping is only happening at the top level of the type. In the `Zero` case of this match on unrolled `sub' n'`, the code returns `Zero`. Constructors' return types are applications of signature functors. Here, `Zero` can be given type `Nat (μ Nat)`, which is then coerced to `μ Nat` by `roll`. The function `subs` does similar rolling and unrolling, but also invokes subtraction (`sub`). Since `sub` returns an algebra, the code uses `cata` to coerce that to the function which accepts the subtrahend.

Thanks to the restrictions imposed by Mendler-style algebras, these functions have the desirable property that they are guaranteed to terminate on all inputs. Nevertheless, we expect the reader will agree that the code is rather burdened with coercions. This raises the question we address in this paper: can we support algebraic programming like this with lighter syntax? We return with our positive answer for these examples in Figure 11 below.

4 Typing

TODO Henry: Write prose for introducing type system and explain how it fits into the running examples from "Motivating Examples". Here's the idea: Here are the most interesting typing rules. The rest are in Appendix A.

$$\textsc{Gamma}\ \frac{\overline{\langle D, \mu X.k_i(\overline{A_{ij}\ \times})\ +\rangle \in \mathscr{D}} \qquad a : D\ C \qquad \overline{\overline{\Gamma, x_{ij} : A_{ij}[X \mapsto C],} \vdash b_i : B}}{\Gamma \vdash \gamma a.\{\overline{k_i(\overline{x_{ij},}) \mapsto b_i\ |}\} : B}$$

$$\textsc{Omega}\ \frac{\Gamma, (R_f : *), (f : R_f \to A), (x : D\ R_f) \vdash b : B}{\Gamma \vdash \omega f(x).b : D \Rightarrow B} \qquad \textsc{Subtype}\ \frac{\Gamma \vdash A <: A' \qquad \Gamma \vdash a : A}{\Gamma \vdash a : A'}$$

Fig. 3: Notable typing rules

5 Type checking with constraints

TODO Henry: mention we are not doing full type inference, and are just using type variables for the sake of inferring coercions. so, lambdas and lets still have annotations

We present a type checking procedure based on generating and solving subtyping constraints. First, subtyping constraints are generated from a structurally recursive analysis of a term, where each constraint corresponds to a position in the term at which a coercion (which may end up being the identity) is required to justify the subtype relation. Then, these subtyping constraints are solved by a logic-programming engine by applying the algorithmic subtyping rules in Figure 7, where each rule corresponds to a constructor of the coercion to be inserted at the constraint's associated position in the term.

$$\textsc{Var}\ \frac{(x : A) \in \Gamma}{\Gamma \vdash x : A \mid \varnothing}$$

$$\textsc{Con}\ \frac{X \text{ fresh} \qquad \langle D, \mu Y. \cdots + k(\overline{A'_i\ \times}) + \ldots \rangle \in \mathscr{D} \qquad \overline{\Gamma \vdash a_i : A_i \mid \mathcal{C}_{a_i}}}{\Gamma \vdash k(\overline{a_i,}) : D\ Y \mid \bigcup_i (\mathcal{C}_{a_i} \cup \{A_i \underset{a_i}{<::} A'_i[Y \mapsto X]\})}$$

$$\textsc{Lam}\ \frac{\Gamma, (x : A) \vdash b : B \mid \mathcal{C}_b}{\Gamma \vdash \lambda(x : A).b : A \to B \mid \mathcal{C}_b} \qquad \textsc{App}\ \frac{X \text{ fresh} \qquad \Gamma \vdash f : B \mid \mathcal{C}_f \qquad \Gamma \vdash a : A \mid \mathcal{C}_a}{\Gamma \vdash f\ a : X \mid \mathcal{C}_f \cup \mathcal{C}_a \cup \{B \underset{f}{<::} A \to X\}}$$

$$\textsc{Gamma}\ \frac{\begin{array}{c} X, Y \text{ fresh} \qquad \dfrac{\langle D, \mu X. \overline{k_i(\overline{A_{ij}\ \times})\ +} \rangle \in \mathscr{D}}{\Gamma, \overline{x_{ij} : A_{ij}[X \mapsto C]}, \vdash b_i : B_i \mid \mathcal{C}_{b_i}} \\ \Gamma \vdash a : C \mid \mathcal{C}_a \end{array}}{\Gamma \vdash \gamma a.\{\overline{k_i(\overline{x_{ij},}) \mapsto b_i\ |}\} : Y \mid \{C \underset{a}{<::} D\ X\} \cup \mathcal{C}_a \cup \bigcup_i (\{B_i \underset{b_i}{<::} Y\} \cup \mathcal{C}_{b_i})}$$

$$\textsc{Omega}\ \frac{\Gamma, (R_f : *), (f : R_f \to A), (x : D\ R_f) \vdash b : B \mid \mathcal{C}_b}{\Gamma \vdash \omega f(x).b : D \Rightarrow B \mid \mathcal{C}_b}$$

$$\textsc{Let}\ \frac{\Gamma \vdash a : A \mid \mathcal{C}_a \qquad \Gamma, (x : A') \vdash b : B \mid \mathcal{C}_b}{\Gamma \vdash \mathit{let}\ x : A' = a\ \mathit{in}\ b : B \mid \mathcal{C}_a \cup \{A \underset{a}{<::} A'\} \cup \mathcal{C}_b}$$

Fig. 4: Constraint generation rules.

Constraint generation for a term a in context Γ is performed by applying the constraint generation rules in Figure 4 recursively on the structure of a, which upon success produces a derivation of $\Gamma \vdash a : A \mid \mathcal{C}$. This is a usual typing judgment augmented with a constraint problem $\mathcal{C}$, where $\mathcal{C}$ is a set of constraints. Each constraint in these sets is of the form $A <::_{a'} B$, where a' is a specific sub-term of a, including its unique location within a. The system is designed such that a solution to $\mathcal{C}$ specifies an insertion of coercions around sub-terms of a, where each coercion inserted at a position a' corresponds to one of the original required constraints $A <::_{a'} B$.

The constraint generation phase also performs type inference, however in contrast to usual type inference and checking procedures, this procedure cannot decide a term to be mal-typed. The constraint generation procedure will generate a constraint problem for any well-formed and well-scoped Γ, a, A. A derivation $\Gamma \vdash a : A \mid \mathcal{C}$ merely states that a has type A *only if* the constraint problem $\mathcal{C}$ is solvable.

$$\textsc{ReflCoe}\ \frac{}{\Gamma \vdash id[\mu D] : \mu D <:: \mu D} \qquad \textsc{RecVarCoe}\ \frac{}{\Gamma \vdash id[R_f] : R_f <:: R_f}$$

$$\textsc{ArrCoe}\ \frac{\Gamma \vdash c_1 : A' <:: A \qquad \Gamma \vdash c_2 : B <:: B'}{\Gamma \vdash arr\ c_1\ c_2 : A \to B <:: A' \to B'} \qquad \textsc{CovCoe}\ \frac{\Gamma \vdash c : A <:: A'}{\Gamma \vdash cov[D]\ c : D\ A <:: D\ A'}$$

$$\textsc{AlgCoe}\ \frac{\Gamma \vdash c : A <:: A'}{\Gamma \vdash alg[D]\ c : D \Rightarrow A <:: D \Rightarrow A'} \qquad \textsc{RollCoe}\ \frac{\Gamma \vdash c : A <:: \mu D}{\Gamma \vdash roll\ c : D\ A <:: \mu D}$$

$$\textsc{UnrollCoe}\ \frac{\Gamma \vdash c : \mu D <:: A}{\Gamma \vdash unroll\ c : D\ A <:: \mu D} \qquad \textsc{CataCoe}\ \frac{\Gamma \vdash c_1 : B <:: D \qquad \Gamma \vdash c_2 : A <:: C}{\Gamma \vdash cata\ c_1\ c_2 : D \Rightarrow A <:: B \to C}$$

Fig. 5: Coercion typing rules.

For example, constraint generation for `sub : Nat → Nat ⇒ Nat` results in the following constraint problem, where X, Y, Z, W, U are fresh type variables and R is a fresh recursion type variable:

TODO Henry: Make sure to update this if the constraint generation rules change.

TODO Henry: Insert term positions on constraints, just using the terms themselves

$$\{X \to Nat \Rightarrow U <:: \mu Nat \to Nat \Rightarrow \mu Nat, Nat\ Y <:: Nat\ W, R \to \mu Nat <:: R \to$$

$$Nat\ Y, Nat\ Z <:: U, \mu Nat <:: U\}$$

Here is where the constraints are introduced.

- The requirement $X \to Nat \Rightarrow U <:: \mu Nat \to Nat \Rightarrow \mu Nat$ is introduced by $\gamma\ n$.
- **TODO** explain sources of a few other requirements – just focus on like 3

TODO Henry: Explain anything else necessary for understanding constraint generation

TODO Henry: State completeness and soundness for the constraint generation rules relative to the typing rules

6 Subtyping

We begin with the declarative subtyping rules corresponding to the coercions of Figure 5, and then propose an algorithmic version, where all meta-variables in the premises of the rule are already present in the conclusion, and the sizes of the

constraints decrease. This ensures that SLD resolution will not diverge on ground constraints. We prove that the two sets of rules are equivalent.

6.1 Declarative subtyping

$$\textsc{Refl}\ \frac{}{\mu D <: \mu D} \qquad \textsc{Arr}\ \frac{A' <: A \quad B <: B'}{A \to B <: A' \to B'} \qquad \textsc{Cov}\ \frac{A <: A'}{D\ A <: D\ A'} \qquad \textsc{Alg}\ \frac{A <: A'}{D \Rightarrow A <: D \Rightarrow A'}$$

$$\textsc{Roll}\ \frac{}{D\ \mu D <: \mu D} \qquad \textsc{Unroll}\ \frac{}{\mu D <: D\ \mu D} \qquad \textsc{Cata}\ \frac{}{D \Rightarrow A <: \mu D \to A} \qquad \textsc{Trans}\ \frac{A <: A' \quad A' < A''}{A <: A''}$$

Fig. 6: Declarative subtyping rules.

Figure 6 shows the rules for declarative subtyping $A <: B$. We have reflexivity rule Refl for datatypes. Arr is the usual subtyping rule for function types, which is contravariant in the domain and covariant in the codomain. Cov expresses that the signature functor is indeed a (covariant) functor. Alg expresses that algebra types $D \Rightarrow A$ are covariant in their carriers A. Roll and Unroll together express that μF is equivalent to $F\ \mu F$. Cata expresses that an algebra can be coerced to a function, namely the algebra's catamorphism. Finally, there is the Trans rule, completing the definition of subtyping as a partial order.

6.2 Algorithmic subtyping

$$\textsc{Refl}\ \frac{}{\mu D <:: \mu D} \qquad \textsc{Arr}\ \frac{A' <:: A \quad B <:: B'}{A \to B <:: A' \to B'} \qquad \textsc{Cov}\ \frac{A <:: A'}{D\ A <:: D\ A'}$$

$$\textsc{Alg}\ \frac{A <:: A'}{D \Rightarrow A <:: \mu D \Rightarrow A'} \qquad \textsc{Roll}\ \frac{A <:: \mu D}{D\ A <:: \mu D} \qquad \textsc{Unroll}\ \frac{\mu D <:: A}{\mu D <:: D\ A} \qquad \textsc{Cata}\ \frac{B <:: \mu D \quad A <:: C}{D \Rightarrow A <:: B \to C}$$

Fig. 7: Algorithmic subtyping rules.

Figure 7 presents the rules for algorithmic subtyping $A <:: B$. We use similar names as for the declarative rules, but critically, the algorithmically problematic Trans rule is not present. A consequence of this is that now several rules that did not have

premises in the declarative formulation, here do. For example, compare the declarative (on the left) and algorithmic CATA rules:

$$\frac{}{D \Rightarrow A <: \mu D \to A} \qquad \frac{B <:: \mu D \quad A <:: C}{D \Rightarrow A <:: B \to C}$$

The algorithmic rule needs to take into account subtyping relationships between the components in the conclusion, because there is no rule for transitivity, which could allow those relationships to be taken into account after this rule is applied. In effect, transitivity needs to be built into all the other rules. We see a similar change from the declarative to the algorithmic versions of the ROLL and UNROLL rules, for rolling and unrolling the signature functor.

Lemma 1 (Decrease) *For all rules of Figure 7, the sizes of the constraints in the premises are less than the size of the constraint in the conclusion.*

Theorem 2 (Decidability) *It is decidable whether or not two types are in the algorithmic subtyping relation.*

Proof Bottom-up application of the rules to ground types is guaranteed to terminate, by Lemma 1. Therefore, proof search may be used as a decision procedure for algorithmic subtyping of ground types. □

6.3 Equivalence of declarative and algorithmic subtyping

While Theorem 2 ensures that we can test ground subtyping constraints using proof search, we must confirm that the algorithmic rules are indeed a correct version of the declarative ones. For this, we prove the following theorems.

Theorem 3 (Soundness) *$A <:: B$ implies $A <: B$.*

Theorem 4 (Completeness) *$A <: B$ implies $A <:: B$.*

The sets of rules are quite similar, so these proofs are not onerous. They do depend on the following inductively proven lemmas.

Lemma 5 (roll1 (declarative)) *If $A <: \mu D$, then $D\ A <: \mu D$.*

Lemma 6 (roll2 (declarative)) *If $\mu D <: A$, then $\mu D <: D\ A$.*

Lemma 7 (refl (algorithmic)) *$A <:: A$.*

```
data Tp : Set where
  D_ : Tp → Tp
  _⟶_ : Tp → Tp → Tp
  D⇒ : Tp → Tp
  μD : Tp

infixr 9 _⟶_
infixr 10 D_
```

Fig. 8: Agda definition of types.

Lemma 8 (trans (algorithmic)) *If* $A <:: B$ *and* $B <:: C$ *then* $A <:: C$.

Lemma 9 (roll (algorithmic)) $D\ \mu D <: \mu D$

Lemma 10 (unroll (algorithmic)) $\mu D <: D\ \mu D$

6.4 Formalization in Agda

Types are represented by the Agda datatype `Ty` in Figure 8. For simplicity, we have a single signature functor `D` and its least fixed-point μ`D` (note that this is a single identifier in Agda, not an application of a μ operator to `D`). Algebra types $D \Rightarrow A$ are represented just as `D⇒ A`. Note that `D⇒` is also a single symbol in Agda. Arrow types are represented $A \longrightarrow B$ (as other arrow notations are already in use). The figure declares some fixity information, for lighter notation below.

Figure 9 show the representation of the algorithmic subtyping system; we omit a similar definition for declarative subtyping, for space reasons. Each set of rules is represented as an indexed inductive type, with each rule a constructor. For example, we have, for both types, `Refl` of type μ`D <:` μ`D`, which corresponds exactly to the `Refl` rules of Figures 6 and 7. Conveniently, Agda allows reusing constructor names across datatypes. Dependent typing is used for quantification. For example, the `Roll` constructor has type

```
{T : Tp} → T <:: μD → D T <:: μD
```

This is a dependent function type, stating that give `T : Tp` and a proof of `T <::` μ`D`, `Roll` returns a proof of `D T <::` μ`D`. Finally, the statements of soundness and completeness are shown in Figure 10. The total number of lines for all the Agda code is under 150, a very modest burden of proof. This is partly because types do not contain bound variables, and so the technical overhead associated with these (for example, with de Bruijn indices) is avoided.

```
infix 5 _<::_

data _<::_ : Tp → Tp → Set where
  Refl : μD <:: μD
  Arr : {T1 T2 T1' T2' : Tp} →
         T1' <:: T1 →
         T2 <:: T2' →
         T1 ⟶ T2 <:: T1' ⟶ T2'
  Roll : {T : Tp} → T <:: μD → D T <:: μD
  Unroll : {T : Tp} → μD <:: T → μD <:: D T
  Cov : {T1 T2 : Tp} → T1 <:: T2 → D T1 <:: D T2
  Alg : {T1 T2 : Tp} → T1 <:: T2 → D⇒ T1 <:: D⇒ T2
  Cata : {T T1 T2 : Tp} →
        T1 <:: μD →
        T <:: T2 →
        D⇒ T <:: T1 ⟶ T2
```

Fig. 9: Agda definition of algorithmic subtyping.

```
Soundness    : {T T' : Tp} → T <:: T' → T <: T'
Completeness : {T T' : Tp} → T <: T'  → T <:: T'
```

Fig. 10: Agda statements of soundness and completeness.

6.5 Constraint solving with Chronolog

We take a logic-programming approach to solving problems $\mathcal{C}$ in such a way that calculates the coercions that justify the generated subtyping relations. Our implementation uses a logic-programming engine, Chronolog, which adopts an alternative technique for constraint suspension compared to rule-level techniques in Prolog variants (e.g. Prolog II). **TODO** source: previous work on delayed constraints in Prolog-style logic PLs Chronolog accepts a set of constraints, a logic variable substitution, and a suspension meta-predicate S. Anytime a constraint that satisfies S appears, that constraint is immediately suspended. Suspended constraints are still refined by logic variable substitutions. Chronolog can terminate in three ways: (1) failure with unsolved constraints, (2) progress with suspended constraints, (3) success with no suspended constraints.

A particular choice of S is critical in this context due to the presence of the Roll and Unroll rules, which can be composed infinitely by straightforward SLD resolution, violating completeness of type checking. Chronolog prevents such infinite regresses by supporting a global suspension meta-predicate, S, where constraints that satisfy S are immediately suspended before being processed further. If such a refinement results in a constraint no longer satisfying S, then the constraint is resumed. We use a meta-predicate for S that holds of these forms of constraints, where X, Y are type variables, D is a datatype name, R is a recursion type variable, and A, B are

types.

$$X <:: \mu D, X <:: D\ A, X <:: R, \mu D <:: X, D\ A <:: X, R <:: X, X <:: Y$$

Suspending constraint of these forms prevents Chronolog from applying rules in a cycle, which follows from a case analysis of the all possible rule applications to constraints not of suspended forms. In particular, the only forms of constraints that have a type variable as one side of the relation that are not suspended are those were the other side of the relation is either $D \Rightarrow A$ or $A \to B$. This allows Chronolog, for example, to attempt to solve constraints of the form $X <:: A \to B$ with both Arr and Cata, or to attempt to solve constraints of the form $A \Rightarrow B <:: X$ with both Alg and Cata.

TODO Henry: mention that Solve is nondeterministic

Algorithm 1 Constraint solving with Chronolog.

```
1:  function Solve(C)
2:      σ ← ∅
3:      while C ≠ ∅ do
4:          result ← Chronolog(S, C, σ)
5:          match result with
6:            case ⟨"failure"⟩ ⇒ return ⊥
7:            case ⟨"success", σ'⟩ ⇒ return σ'
8:            case ⟨"progress", σ', C'⟩ ⇒
9:              constraint ← Sample(C')
10:             σ ← Refine(constraint) ∘ σ'
11:             C ← σ(C')
12:         end match
13:     end while
14: end function
```

Constraints solving is accomplished by using Chronolog inside a loop that processes any suspended constraints left over after each iteration, as shown in Algorithm 1. Each iteration, a single suspended constraint is chosen to be processed by Refine, resulting in a substitution that refines that suspended constraint (and may incidentally refine other suspended constraints). The choice of which suspended constraint to process may affect the order in which Chronolog explores solutions but all choices lead to equivalent solutions up to permutations of Roll and Unroll (if there is a solution).
TODO Henry: justify this statement

$$\begin{aligned}
\textsc{Refine}(\mu D <:: X) &= \textsc{Refine}(X <:: \mu D) &&= \{X \mapsto \mu D\} \\
\textsc{Refine}(D\ A <:: X) &= \textsc{Refine}(X <:: D\ A) &&= \{X \mapsto D\ Y\} \text{ where } Y \text{ fresh} \\
\textsc{Refine}(R <:: X) &= \textsc{Refine}(X <:: R) &&= \{X \mapsto R\} \\
&\quad\ \textsc{Refine}(X <:: Y) &&= \{X \mapsto Y\}
\end{aligned}$$

Lemma 11 (Termination) *For any constraint problem $\mathcal{C}$,* Solve($\mathcal{C}$) *terminates.*

Proof We show termination by demonstrating a strictly decreasing metric over each iteration of the loop in Solve. Each iteration begins with an invocation of Chronolog. If Chronolog results in $\langle$"failure"$\rangle$ or $\langle$"success"$\rangle$, then we are done. If Chronolog results in $\langle$"progress", $\sigma', \mathcal{C}'\rangle$, then by Lemma 1 the leftover $\mathcal{C}'$ must be in total smaller than the original $\mathcal{C}$ from the beginning of this iteration.

The substitution resulting from Refine(*constraint*) preserves the size of constraints in all but one case. When Refine(*constraint*) increases the size of constraints in one case, where it results in $\{X \mapsto D\ Y\}$, Chronolog will strictly decrease the size of these constraints again immediately, as follows from a case analysis on all the possible results of applying Refine(*constraint*) to suspended constraints. □

Lemma 12 (Completeness) *If σ is a solution to a constraint problem $\mathcal{C}$, then $\exists \sigma' \in [\![\sigma]\!]_\sim$ such that $\sigma' \in$* Solve($\mathcal{C}$).

Proof Solve will finds all solutions (up to permutations of Roll and Unroll) to $\mathcal{C}$ because Chronolog attempts every valid application of the algorithmic subtyping rules which are complete by Theorem 4, Refine preserves validity of solution σ, and Solve terminates by theorem 11. □

While there may be multiple solutions to $\mathcal{C}$, solutions to such constraint problems generated by the procedure described in Section 5 are equivalent up permutations of Roll and Unroll **TODO** Henry: justify this claim. Furthermore, solutions related by this equivalence, when applied as coercion insertions to terms, result in semantically equivalent terms where `roll` and `unroll` are semantically interpreted as identity functions **TODO** Henry: justify this claim.

Algorithm 2 Type checking for $\mathcal{L}_{cata}$.

```
1: function TypeCheck(Γ, a, A)
2:     C ← Generate(Γ, a, A)
3:     match Solve(C) with
4:         case [ ] ⇒ ⊥
5:         case _ ⇒ ⊤
6:     end match
7: end function
```

Theorem 13 *If $\Gamma \vdash a : A$ according to the declarative typing rules for $\mathcal{L}_{cata}$ displayed in Figure 3, then* TypeCheck(Γ, a, A) *(Algorithm 2) accepts.*

Proof **TODO** Henry: Essentially, by Theorem 4 and Lemma 12. □

7 Returning to the Examples

Recall the examples from Section 3 where `sub` and `subs` were implemented algebraically. Explicit coercions using `unroll`, `roll`, and `cata` were required to operate with the representation of datatypes as fixed points of signature functors and the functional interpretation of algebras via a catamorphism. Now, with the machinery of coercive subtyping in place, we can write those examples without any coercions, as demonstrated in Figured 11.

```
sub : μ Nat → Nat ⇒ μ Nat
sub = λ (m : μ Nat) . ω sub'(n) . γ n .
  { Zero → m
  | Suc(n') → γ sub' n' .
    { Zero → Zero
    | Suc(m') → m' } }

subs : NatList ⇒ μ NatList → μ NatList
subs = ω subs'(l1) . λ (l2 : μ NatList) . γ l1 .
  { Nil → Nil
  | Cons(h1, t1) → γ l2 .
    { Nil → Nil
    | Cons(h2, t2) →
        Cons(sub h1 h2, subs' t1 t2) } }
```

Fig. 11: Code for `sub` and `subs` with coercions all implicit.

The type checking procedure described in previous sections collects and solves the implied subtyping constraints, which determine an insertion of coercions into these terms which matches the manual usage of explicit coercions in the original examples.

8 Related Work

Turner introduced the term *strong functional programming*, presented several arguments in favor of the approach, and proposed using structural termination as an *elementary* way to realize such a language [4]. Mendler proposed a recursor using an abstract type to ensure that recursive calls are permitted only on subdata for an inductive type [5]. The algebraic approach has been applied for modular implementation of interpreters [6]. Charity is a strong functional programming language based on categorical abstractions for initial algebras and final coalgebras [7]. Tofu uses the Calculus of Constructions as a language for defining terminating functions, again based on categorical ideas [8]. Charity and Tofu both support coinductive datatypes, which we did not study here.

9 Conclusion and future work

Total algebraic programming can be made more feasible by inferring the basic coercions necessary for the method: rolling and unrolling signature functors, and turning an algebra into its catamorphism. Type inference generates subtyping constraints, whose solutions correspond to the needed coercions. A logic-programming engine processes constraints with respect to an algorithmic version of the subtyping rules. This algorithmic subtyping relation is proved equivalent to the declarative one, in Agda.

Future work includes richer forms of algebraic programming. One powerful generalization of structural recursion is to recursive coalgebras [9]. This encompasses divide-and-conquer algorithms, which are beyond the scope of structural recursion. A different approach to divide-and-conquer recursion has also been considered by Abreu et al. [10]. There, algebras call functions to pass between various abstract types. These would be excellent candidates for inference using coercive subtyping, as we proposed in this paper. It would also be interesting to infer coercions for coalgebraic programming with coinductive types.

A Typing rules

Section 4 presented a sampling from the full set of typing rules for $\mathcal{L}_{cata}$ below.

$$\textsc{Var}\ \frac{(x : A) \in \Gamma}{\Gamma \vdash x : A} \qquad \textsc{Con}\ \frac{\langle D, \mu X. \cdots + k(\overline{A_i\ X}) + \cdots \rangle \in \mathscr{D} \qquad \overline{\Gamma \vdash a_i : A_i}}{\Gamma \vdash k(\overline{a_i,}) : D\ X}$$

$$\textsc{Lam}\ \frac{\Gamma, (x : A) \vdash b : B}{\Gamma \vdash \lambda(x : A).b : A \to B} \qquad \textsc{App}\ \frac{\Gamma \vdash f : A \to B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B}$$

$$\textsc{Omega}\ \frac{\Gamma, (R_f : *), (f : R_f \to A), (x : D\ R_f) \vdash b : B}{\Gamma \vdash \omega f(x).b : D \Rightarrow B}$$

$$\textsc{Gamma}\ \frac{\overline{\langle D, \mu X. k_i(\overline{A_{ij}\ X}) + \rangle \in \mathscr{D}} \qquad a : D\ C \qquad \overline{\overline{\Gamma, \overline{x_{ij} : A_{ij}[X \mapsto C]}, \vdash b_i : B}}}{\Gamma \vdash \gamma a.\{\overline{k_i(\overline{x_{ij},}) \mapsto b_i\ |}\} : B}$$

$$\textsc{Let}\ \frac{\Gamma, (x : A) \vdash b : B \qquad \Gamma \vdash a : A}{\Gamma \vdash \mathit{let}\ x : A = a\ \mathit{in}\ b : B} \qquad \textsc{Subtype}\ \frac{\Gamma \vdash A <: A' \qquad \Gamma \vdash a : A}{\Gamma \vdash a : A'}$$

References

[1] Uustalu, T., Vene, V.: Mendler-style Inductive Types, Categorically. Nordic J. of Computing **6**(3), 343–361 (1999)

[2] Hagino, T.: A Categorical Programming Language. PhD thesis, University of Edinburgh (1987)

[3] Goguen, J.A., Thatcher, J.W., Wagner, E.G., Wright, J.B.: Initial algebra semantics and continuous algebras. J. ACM **24**(1), 68–95 (1977) https://doi.org/10.1145/321992.321997

[4] Turner, D.A.: Elementary Strong Functional Programming. In: Proceedings of the First International Symposium on Functional Programming Languages in Education. FPLE '95, pp. 1–13. Springer, Berlin, Heidelberg (1995)

[5] Mendler, N.P.: Inductive types and type constraints in the second-order lambda calculus. Annals of Pure and Applied Logic **51**(1), 159–172 (1991)

[6] Swierstra, W.: Data Types à La Carte. J. Funct. Program. **18**(4), 423–436 (2008)

[7] Cockett, R., Fukushima, T.: About Charity. Yellow Series Report No. 92/480/18, Department of Computer Science, The University of Calgary (June 1992)

[8] Widemann, B.T.: Tofu - towards practical church-style total functional programming. In: Workshop der GI-Fachgruppe Programmiersprachen und Rechenkonzepte, Bad Honnef, 3.-5. Mai (2010). Available at https://www-ps.informatik.uni-kiel.de/fg214/Honnef2010/TechnischerBericht1010.pdf

[9] Capretta, V., Uustalu, T., Vene, V.: Recursive coalgebras from comonads. Inf. Comput. **204**(4), 437–468 (2006) https://doi.org/10.1016/J.IC.2005.08.005

[10] Abreu, P., Delaware, B., Hubers, A., Jenkins, C., Morris, J.G., Stump, A.: A type-based approach to divide-and-conquer recursion in coq. Proc. ACM Program. Lang. **7**(POPL), 61–90 (2023) https://doi.org/10.1145/3571196